

COMM114: Generative AI - GAN (part 2)

Guoqiang Zhang

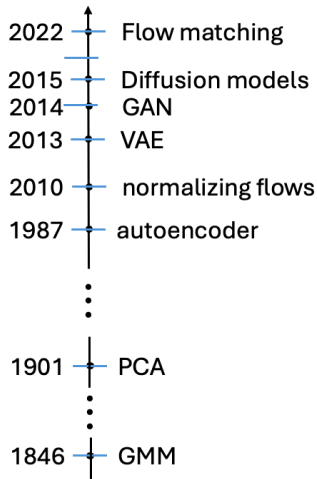
October 2025

Table of Contents

- Wasserstein GAN (WGAN)
- WGAN with Gradient Penalty (WGAN-GP)
- GAN Architectures
- Applications

Recap from Last Lecture

Generative adversarial network (GAN) was invented in 2014 by Ian Goodfellow and his colleagues in June 2014.



Recap from Last Lecture

The standard GAN updates the generator and discriminator by optimizing the cross-entropy:

$$\arg \min_{\phi} \max_{\theta} \left[\underbrace{\mathbb{E}_{\mathbf{x} \sim p_{data}} [\log D_{\theta}(\mathbf{x})]}_{\text{1st term}} + \underbrace{\mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} \log(1 - D_{\theta}(G_{\phi}(\mathbf{z})))}_{\text{2nd term}} \right]$$

Wasserstein GAN (WGAN)

- Main objective of WGAN: To improve the training stability of GAN.

Wasserstein Generative Adversarial Networks

Martin Arjovsky, Soumith Chintala, Léon Bottou Proceedings of the 34th International Conference on Machine Learning, PMLR 70:214-223, 2017.

Abstract

We introduce a new algorithm named WGAN, an alternative to traditional GAN training. In this new model, we show that we can improve the stability of learning, get rid of problems like mode collapse, and provide meaningful learning curves useful for debugging and hyperparameter searches. Furthermore, we show that the corresponding optimization problem is sound, and provide extensive theoretical work highlighting the deep connections to different distances between distributions.

Wasserstein GAN (WGAN)

- To be differentiable from the standard GAN, we denote the discriminator of WGAN as $f_{\theta}(\cdot)$.
- WGAN makes use of the Wasserstein distance (alternatively referred to as **earth mover distance (EMD)**) to replace the cross-entropy of the standard GAN.

$$\begin{aligned} & \arg \min_{\phi} \max_{\theta} W(\theta, \phi) \\ &= \arg \min_{\phi} \max_{\theta} W(\theta, \phi) = \underbrace{\left(\max_{\|\nabla f\| \leq 1} \left[\mathbb{E}_{\mathbf{x} \sim p_{data}} [f_{\theta}(\mathbf{x})] - \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [f_{\theta}(G_{\phi}(\mathbf{z}))] \right] \right)}_{\text{Wasserstein distance}} \quad (1) \end{aligned}$$

Remark: The Wasserstein distance requires the function f in the discriminator to be sufficiently smooth (i.e., $\|\nabla f\| \leq 1$). Intuitively, the function $f(\cdot)$ does not change too much when a perturbation is introduced in the input.

Wassserstein GAN (WGAN)

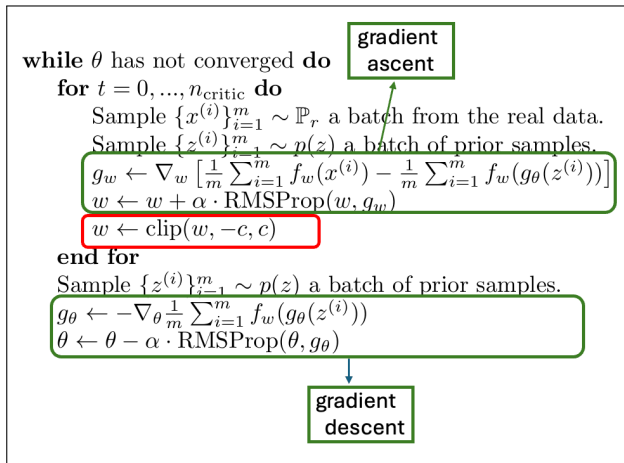
Question:

How to enforce the discriminator $f_{\theta}(\cdot)$ to satisfy the constraint $\|\nabla f\| \leq 1$ in (1) during the training process?

- Difficult to enforce the exact constraint (**not practical**).
- Approximation
 - weight clipping (exploited in the WGAN paper, which is a very rough approximation)
 - Gradient Penalty (WGAN-GP)

Weight Clipping in WGAN

- Basic Idea: A neural network with bounded weights is smooth.
- Pros:
 - Fast to perform weight clipping
 - Simple to implement
- Cons:
 - A very rough approximation of the constraint $\|\nabla f\| \leq 1$



Remark: The figure is from the original paper "Wasserstein Generative Adversarial Networks". See next slide for the training procedure for optimizing (1).

Training procedure

Algorithm 1 Training of WGAN

- 1: **for** number of training iterations **do**
- 2: **for** for k steps **do**
- 3: Sample minibatch of m noise samples $\{z_1, \dots, z_m\}$ from $\mathcal{N}(z; \mathbf{0}, \mathbf{I})$
- 4: Sample minibatch of m real data samples $\{x_1, \dots, x_m\}$
- 5: Update the discriminator via SGD:

$$\theta \leftarrow \theta + \eta \nabla_{\theta} \frac{1}{m} \sum_{i=1}^m \left[f_{\theta}(x_i) - f_{\theta}(G_{\theta}(z_i)) \right]$$

- 6: $\theta \rightarrow \text{clip}(\theta, -c, c)$
- 7: **end for**
- 8: Sample minibatch of m noise samples $\{z_1, \dots, z_m\}$ from $\mathcal{N}(z; \mathbf{0}, \mathbf{I})$
- 9: Update the generator via SGD:

$$\phi \leftarrow \phi - \eta \nabla_{\phi} \frac{1}{m} \sum_{i=1}^m \left[-f_{\theta}(G_{\phi}(z_i)) \right]$$

- 10: **end for**
-

Remark: code in blue is for updating generator while code in red is for updating discriminator.

A Note on WGAN and Standard GAN

Question:

How is $D_{\theta}(\cdot)$ in the standard GAN related to $f_{\theta}(\cdot)$ in WGAN?

- In the standard GAN, the output $D_{\theta}(\cdot)$ is the probability value within $[0, 1]$ indicating the probability of an input sample is real (i.e., the value is close to 1) or fake (e.g., the value is close to 0).
- In the discriminator of the standard GAN, the sigmoid function $\sigma(\cdot)$ is utilized to produce the probability value.
- One can simply let $D_{\theta}(x) = \sigma(f_{\theta}(x))$ to convert the discriminator $f_{\theta}(\cdot)$ of the WGAN to the discriminator $D_{\theta}(\cdot)$ of the standard GAN.

Gradient penalty in WGAN (WGAN-GP)

WGAN-GP is proposed to improve the training of WGAN by avoiding the weight-clipping operation.

Improved Training of Wasserstein GANs

Ishaan Gulrajani¹, Faruk Ahmed¹, Martin Arjovsky², Vincent Dumoulin¹, Aaron Courville^{1,3}

¹ Montreal Institute for Learning Algorithms

² Courant Institute of Mathematical Sciences

³ CIFAR Fellow

igul222@gmail.com

{faruk.ahmed, vincent.dumoulin, aaron.courville}@umontreal.ca
ma4371@nyu.edu

Abstract

Generative Adversarial Networks (GANs) are powerful generative models, but suffer from training instability. The recently proposed Wasserstein GAN (WGAN) makes progress toward stable training of GANs, but sometimes can still generate only poor samples or fail to converge. We find that these problems are often due to the use of weight clipping in WGAN to enforce a Lipschitz constraint on the critic, which can lead to undesired behavior. We propose an alternative to clipping weights: penalize the norm of gradient of the critic with respect to its input. Our proposed method performs better than standard WGAN and enables stable training of a wide variety of GAN architectures with almost no hyperparameter tuning, including 101-layer ResNets and language models with continuous generators. We also achieve high quality generations on CIFAR-10 and LSUN bedrooms. [†]

WGAN-GP

- WGAN-GP is designed by introducing a gradient penalty in WGAN and removing weight-clipping.

$$\begin{aligned} & \arg \min_{\phi} \max_{\theta} \tilde{W}(\theta, \phi) \\ &= \arg \min_{\phi} \max_{\theta} \left(\left[\mathbb{E}_{\mathbf{x} \sim p_{data}} [f_{\theta}(\mathbf{x})] - \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [f_{\theta}(G_{\phi}(\mathbf{z}))] \right] \right. \\ & \quad \left. + \lambda \mathbb{E}_{\tilde{\mathbf{x}} = \epsilon \mathbf{x} + (1-\epsilon) G_{\phi}(\mathbf{z})} (\|\nabla_{\tilde{\mathbf{x}}} f_{\theta}(\tilde{\mathbf{x}})\|_2 - 1)^2 \right) \end{aligned} \quad (2)$$

- Pros:
 - Simple to implement by adding a loss
- Cons:
 - Parameter λ has to be tuned for good performance.

WGAN-GP (cont.)

Impact of gradient penalty in WGAN-GP:

- The gradient penalty prevents the occurrence of extremely large gradients when training the discriminator. As a result, it reduces the risk of mode collapse and improves the quality of the resulting synthetic data.

WGAN-GP (cont.)

pytorch code for gradient penalty ([website link for source code](#))

```
def compute_gp(netD, real_data, fake_data):
    batch_size = real_data.size(0)
    # Sample Epsilon from uniform distribution
    eps = torch.rand(batch_size, 1, 1, 1).to(real_data.device)
    eps = eps.expand_as(real_data)

    # Interpolation between real data and fake data.
    interpolation = eps * real_data + (1 - eps) * fake_data

    # get logits for interpolated images
    interp_logits = netD(interpolation)
    grad_outputs = torch.ones_like(interp_logits)

    # Compute Gradients
    gradients = autograd.grad(
        outputs=interp_logits,
        inputs=interpolation,
        grad_outputs=grad_outputs,
        create_graph=True,
        retain_graph=True,
    )[0]

    # Compute and return Gradient Norm
    gradients = gradients.view(batch_size, -1)
    grad_norm = gradients.norm(2, 1)
    return torch.mean((grad_norm - 1) ** 2)
```

GAN Architectures

Deep Convolutional Generative Adversarial Network (DCGAN)

[cs.LG] 7 Jan 2016

UNSUPERVISED REPRESENTATION LEARNING WITH DEEP CONVOLUTIONAL GENERATIVE ADVERSARIAL NETWORKS

Alec Radford & Luke Metz

indico Research

Boston, MA

{alec,luke}@indico.io

Soumith Chintala

Facebook AI Research

New York, NY

soumith@fb.com

ABSTRACT

In recent years, supervised learning with convolutional networks (CNNs) has seen huge adoption in computer vision applications. Comparatively, unsupervised learning with CNNs has received less attention. In this work we hope to help bridge the gap between the success of CNNs for supervised learning and unsupervised learning. We introduce a class of CNNs called deep convolutional generative adversarial networks (DCGANs), that have certain architectural constraints, and demonstrate that they are a strong candidate for unsupervised learning. Training

GAN Architectures (cont.)

Generator architecture of DCGAN:

```
class Generator(nn.Module):
    def __init__(self, ngpu):
        super(Generator, self).__init__()
        self.ngpu = ngpu
        self.main = nn.Sequential(
            # input is Z, going into a convolution
            nn.ConvTranspose2d( nz, ngf * 8, 4, 1, 0, bias=False),
            nn.BatchNorm2d(ngf * 8),
            nn.ReLU(True),
            # state size. ``(ngf*8) x 4 x 4``
            nn.ConvTranspose2d(ngf * 8, ngf * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf * 4),
            nn.ReLU(True),
            # state size. ``(ngf*4) x 8 x 8``
            nn.ConvTranspose2d( ngf * 4, ngf * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf * 2),
            nn.ReLU(True),
            # state size. ``(ngf*2) x 16 x 16``
            nn.ConvTranspose2d( ngf * 2, ngf, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf),
            nn.ReLU(True),
            # state size. ``(ngf) x 32 x 32``
            nn.ConvTranspose2d( ngf, nc, 4, 2, 1, bias=False),
            nn.Tanh()
            # state size. ``(nc) x 64 x 64``
        )

    def forward(self, input):
        return self.main(input)
```

[website link for DCGAN](#)

GAN Architectures (cont.)

Discriminator architecture of DCGAN:

```
class Discriminator(nn.Module):
    def __init__(self, ngpu):
        super(Discriminator, self).__init__()
        self.ngpu = ngpu
        self.main = nn.Sequential(
            # input is ``(nc) x 64 x 64``
            nn.Conv2d(nc, ndf, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            # state size. ``(ndf) x 32 x 32``
            nn.Conv2d(ndf, ndf * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 2),
            nn.LeakyReLU(0.2, inplace=True),
            # state size. ``(ndf*2) x 16 x 16``
            nn.Conv2d(ndf * 2, ndf * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 4),
            nn.LeakyReLU(0.2, inplace=True),
            # state size. ``(ndf*4) x 8 x 8``
            nn.Conv2d(ndf * 4, ndf * 8, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 8),
            nn.LeakyReLU(0.2, inplace=True),
            # state size. ``(ndf*8) x 4 x 4``
            nn.Conv2d(ndf * 8, 1, 4, 1, 0, bias=False),
            nn.Sigmoid()
        )

    def forward(self, input):
        return self.main(input)
```

[website link for DCGAN](#)

GAN Architectures (cont.)

A Style-Based Generator Architecture for Generative Adversarial Networks (StyleGAN)

A Style-Based Generator Architecture for Generative Adversarial Networks

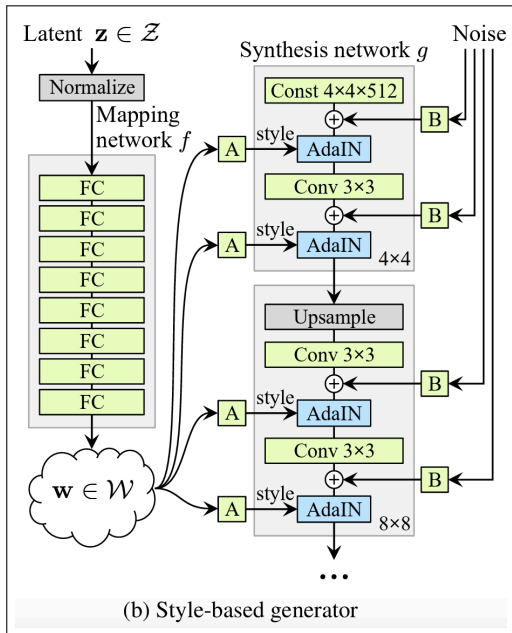
Tero Karras, Samuli Laine, Timo Aila

We propose an alternative generator architecture for generative adversarial networks, borrowing from style transfer literature. The new architecture leads to an automatically learned, unsupervised separation of high-level attributes (e.g., pose and identity when trained on human faces) and stochastic variation in the generated images (e.g., freckles, hair), and it enables intuitive, scale-specific control of the synthesis. The new generator improves the state-of-the-art in terms of traditional distribution quality metrics, leads to demonstrably better interpolation properties, and also better disentangles the latent factors of variation. To quantify interpolation

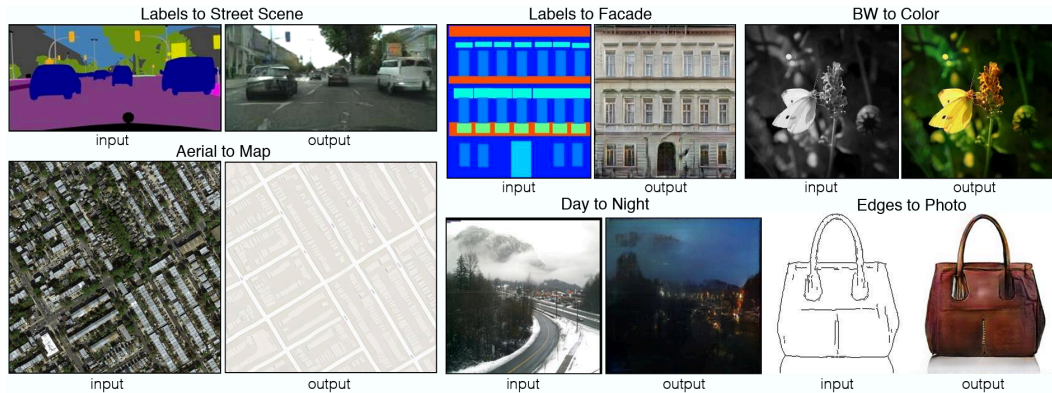
Remark: State-of-the-art GAN models for generating high-quality images.

GAN Architectures (cont.)

- FC: fully connected layer
- Conv: convolutional layer
- AdaIN: adaptive instance normalization
- Const $4 \times 4 \times 512$: a learned tensor beforehand



Applications



Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A. Efros, "Image-to-Image Translation with Conditional Adversarial Networks", 2018.

Applications (cont.)

Fake Face Generator

Remark: Try out the above website to check the quality of the fake face images. **Remark:** The website of "fake face generator" is based on the pre-trained style-GAN model.

Applications (cont.)

Image Editing

Remark: Check the recorded videos on the above website to check how images can be edited. **Remark:** The website of "Image Editing" is based on the pre-trained style-GAN model.

Summary

PGMs	GMMs	VAEs	GANs	NFs	Diffusion Models
Quality			High		
Sampling	Easy	Easy	Easy		
Evaluating $p_{\theta}(x)$	Easy	approximate evaluation	Not possible		

References

- blog of Alexia Jolicoeur-Martineau
- StyleGAN — Official TensorFlow Implementation
- Fake Face Generator
- Lecture notes by Mitesh M. Khapra

Questions?